Causality-Aided Evaluation and Explanation of Large Language Model-Based Code Generation

ZHENLAN JI, Hong Kong University of Science and Technology, China
PINGCHUAN MA*, Hong Kong University of Science and Technology, China
ZONGJIE LI, Hong Kong University of Science and Technology, China
ZHAOYU WANG, Hong Kong University of Science and Technology, China
SHUAI WANG*, Hong Kong University of Science and Technology, China

While code generation has been widely used in various software development scenarios, the quality of the generated code is not guaranteed. This has been a particular concern in the era of large language models (LLMs)-based code generation, where LLMs, deemed a complex and powerful black-box model, are instructed by a high-level natural language specification, namely a *prompt*, to generate code. Nevertheless, effectively evaluating and explaining the code generation capability of LLMs is inherently challenging, given the complexity of LLMs and the lack of transparency.

Inspired by recent progress in causality analysis and its software engineering applications, this paper proposes a causality-driven approach to systematically analyze prompt-code causal relationships. However, this endeavor faces three key technical challenges: (1) representing textual prompts and code in a canonical form, (2) establishing causal relations between high-level concepts and code features, and (3) systematically analyzing diverse prompt variations. To address these challenges, we first propose a novel causal graph-based representation of the prompt and the generated code, which is established over the fine-grained, human-understandable concepts in the input prompts. The formed causal graph is then used to identify the causal relations between the prompt and the derived code. We illustrate the insights that our framework can provide by studying over four popular LLMs with over 12 prompt adjustment strategies. The results of these studies illustrate the potential of our technique to provide insights into LLM effectiveness, and aid end-users in understanding predictions. Additionally, we demonstrate that our approach provides actionable insights to improve the quality of the LLM-generated code by properly calibrating the prompt.

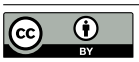
CCS Concepts: • **Software and its engineering → Automatic programming**; • **Computing methodologies → Causal reasoning and diagnostics**.

Additional Key Words and Phrases: Causality, Large Language Models, Code Generation, Explainability

ACM Reference Format:
Zhenlan Ji, Pingchuan Ma, Zongjie Li, Zhaoyu Wang, and Shuai Wang. 2025. Causality-Aided Evaluation and Explanation of Large Language Model-Based Code Generation. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA061 (July 2025), 24 pages. https://doi.org/10.1145/3728938

*Corresponding authors.

Authors' Contact Information: Zhenlan Ji, Hong Kong University of Science and Technology, Hong Kong, China, zjiae@cse.ust.hk; Pingchuan Ma, Hong Kong University of Science and Technology, Hong Kong, China, pmaab@cse.ust.hk; Zongjie Li, Hong Kong University of Science and Technology, Hong Kong, China, zligo@cse.ust.hk; Zhaoyu Wang, Hong Kong University of Science and Technology, Hong Kong, China, zwangjz@cse.ust.hk; Shuai Wang, Hong Kong University of Science and Technology, Hong Kong, China, shuaiw@cse.ust.hk.



ACM 2994-970X/2025/7-ARTISSTA061
https://doi.org/10.1145/3728938

1 Introduction

Code generation serves as one of the most central problems in automated software engineering. It enables machines to program automatically to satisfy human intent expressed in the form of some specification, usually in the form of natural language. The problem of code generation has been studied for decades, and has been applied as the basis of many different important domains, such as program synthesis, program repair, and fuzz testing. In recent years, code generation has achieved great progress in both academia and industry. In particular, we observe that large language models (LLMs) have been applied to support code generation, and have achieved impressive results. For example, GPT-4 has been applied to generate code from natural language, whose coding ability is comparable to that of humans [50]. To date, many LLMs are already seamlessly integrated into the developer's IDE for commercial usage, like GitHub Copilot and Amazon CodeWhisperer, and existing benchmarks [12, 46] further established quantitative metrics for functional correctness.

Despite the great progress, we notice that the quality of the generated code, especially in the era of LLM-based code generation, fluctuates under the variation of natural language specifications. For instance, as will be shown shortly, the way of expressing the same intent in natural language (i.e., prompt) can lead to significantly different code outputs. This aligns with broader NLP findings where minor prompt perturbations affect model outputs [62], but poses unique risks in software contexts. As a result, the behavior of LLM-based code generation is opaque and uninterpretable, which hinders the broader adoption of LLM-based code generation in practice. For instance, it has been reported that GitHub Copilot generates highly unstable code [40] or even generates code containing security vulnerabilities [53].

To date, effectively evaluating and explaining the code generation capability of LLMs is challenging. While the research community has proposed several benchmarks to evaluate the code generation capability of LLMs, such as CodeSearchNet [28] and HumanEval [12], those benchmarks focus primarily on the surface level metrics (e.g., the BLEU score) or functional metrics like the pass rate. However, they fail to capture the interplay between the prompt and the generated code, which is critical to understanding the behavior of LLM-based code generation. While some recent work has proposed approaches to explain the outcome of code generation models [44], they extensively rely on the model's internal mechanics, which are not available for the most common usage (i.e., via API). Efforts to systematize prompt engineering on code generation [80] remain empirical, without causal grounding. Crucially, no existing work systematically analyzes how prompt causally influence code properties. Building on foundational work in causality theory [54] and its emerging software engineering applications [32, 63, 67], we develop the first causal framework for analyzing prompt-code relationships in LLM-based generation.

With the causal framework, our method effectively offers a systematic and human-understandable explanation to LLM-based code generation, by establishing the causal relations between the LLM prompts and the generated code. However, conducting causality analysis in the context of LLM-based code generation is challenging. First, it is difficult to represent the textual prompt and the generated code in a canonical form that is amenable to causality analysis. Second, it is challenging to establish causal relations between high-level concepts in the prompt and code features in the generated code. Third, it is difficult to systematically analyze diverse prompt variations and their impact on the generated code. To address these challenges, we first propose a novel quantification scheme to represent the prompt and the generated code via a number of linguistic features (for prompt) and code features (for generated code). Then, we recast the task of conducting causality analysis between natural language and code as causality analysis among these numerical features. To systematically explore how the diverse form of prompts affects the generated code, we employ an LLM-based rephrasing technique (e.g., "make it longer", "make it more technical") to generate

diverse prompts for the same intent. We then apply a state-of-the-art causality analysis algorithm, namely DiBS [45], to identify the causal relations among these features and estimate the average treatment effect (ATE) of each rephrasing instruction on the generated code. The ATE indicates the average difference of a feature (e.g., BLEU score) in the generated code when the prompt is rephrased (e.g., to be lengthy) compared to the original prompt. We then use the ATE as a building block to enable holistic analysis that can explain the potential causes on a metric of interest (e.g., BLEU score) in the generated code. For instance, we can identify which linguistic features in the prompt that are most likely to cause the LLM to generate high-quality code and those that are most likely to cause the LLM to generate low-quality code. These actionable insights provide firm and handy guidance to developers to tune the prompt to generate high-quality code.

To gauge the effectiveness of our approach, we conduct extensive experiments using four representative LLMs, GPT-Neo [25], GPT-3.5-Turbo (ChatGPT) [5], GPT-4, and Qwen-2.5 [27, 83] This empirical study enables a comprehensive analysis of the trade-offs and leads to a number of intriguing findings over LLM prompts and code outputs. First, as expected, the usage of certain rephrasing instructions may notably affect the pattern of produced code output, highlighting the need for a systematic and automated approach to deciding optimal rephrasing instructions usage for particular scenarios. Second, certain instruction, such as `Fluent` and `Formal`, deliver visible "tradeoffs" in the generated code, which may become a practical obstacle for developers to tune the prompt. Moreover, the reaction of LLMs to prompts with certain properties may be unreasonable, revealing some severe problems in the LLMs (e.g., overfitting to the training data). This observation highlights the importance of taking both human-understandable and other subtle factors into account when calibrating the LLM prompt. Last, we demonstrate a versatile and important application of our framework to improve prompts. With empirical assessment, we show the selected prompts offer superior code generation quality compared to the original prompt, illustrating the potential of our approach in assisting developers in refining the prompt. To conclude, this paper makes the following contributions:

- Given LLM-based code generation, a cornerstone and vastly-used task in software engineering, this paper for the first time uses causality analysis as a principled approach to analyzing how prompts affect this process.
- Technically, we propose a set of domain-specific design considerations to enable accurate and comprehensive causality analysis in the context of LLM-based code generation. These design also aggregates as a novel causal analysis pipeline with high effectiveness and generalizability.
- We conduct extensive evaluations over different LLMs to illustrate the effectiveness and generalizability of our pipeline in code generation quality analysis. A downstream application is also conducted to demonstrate the effectiveness of our approach in improving the quality of the generated code.

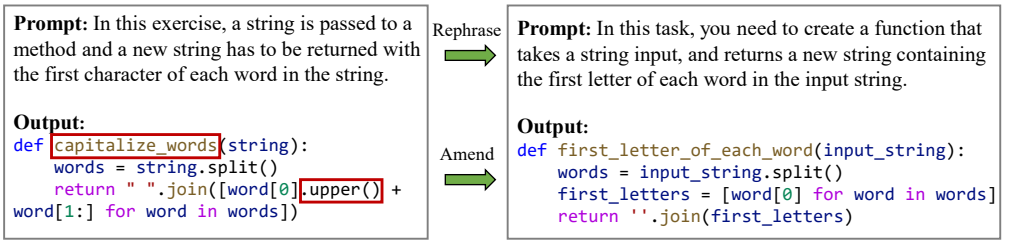
Code and Data. We provide the artifact of this research at [7].

2 Preliminary

2.1 LLM and Prompt Engineering

LLM. In recent years, LLMs have experienced a surge in popularity and adoption across various scenarios owing to their promising performance and high flexibility on a diverse range of tasks. To date, LLMs like ChatGPT [5], which contain over 100 billion parameters, have demonstrated strong capabilities on tasks such as language translation [34], clinical decision [64], and various software engineering tasks [42, 42, 74, 78].

Prompt Engineering. One of the key factors that contributes to the success of LLMs is the input prompt, which is a text or template that provides task-specific knowledge to the model. The



(a) generated code for the original prompt **(b) generated code for the improved, clearer prompt**

Fig. 1. Example of prompt engineering for code generation. Red boxes indicate the error in the code.

exceeding scalability of LLMs facilitates the adoption of "pre-train & prompt," where merely prompts are required to adjust the model for specific downstream tasks, instead of the conventional "pre-train & fine-tune" paradigm. To date, various methods that improve LLM performance via prompts have been proposed, including few-shot learning [11, 60], chain-of-thought [75, 79], and debate LLM [16]. Consequently, the design of prompts spearhead a new research area dubbed "prompt engineering" in LLM research [43]. This technique has also been extended to code generation [41, 74, 80, 81] and we will discuss its effect on code quality in Sec. 3.1. The existing advances, however, relying on manual effort and heuristics due to the lack of theoretical foundations underlying prompt engineering, necessitate a systematic investigation to elucidate how subtle aspects of prompts impact models and then yield insights for crafting high-quality prompts.

2.2 Code Generation

Code generation is a fundamental software engineering technique [24]. Previous research ranged from search-based synthesis [23, 48, 65] to neural network-based generation [25, 39, 46]. This paper focuses on generating code from natural language text, with Python3 as the target programming language for its high popularity [18, 36, 77]. Our approach, however, is generalizable to other settings as shown in Sec. 9. Recent years have witnessed a rising interest in applying LLMs to code generation [9, 40, 78]. Although code generation with LLMs also benefits from prompt engineering, the output quality is prone to being compromised by the unstable nature of prompts (see example in Fig. 1 and our detailed discussion in Sec. 3.1). Moreover, unlike humans who are capable of comprehending statements with subtle errors, the low tolerance of compilers and operating systems for bugs and performance issues further exacerbates the problem of prompt instability. In addition, the complexity of the natural language text that constitutes the prompt (e.g., the unstructured nature of natural language) makes it inherently difficult to interpret the prompt's effect on the generated code. All of these issues motivate us to investigate the relations between the prompt and the output quality of LLMs for code generation.

2.3 Causality Analysis

As a canonical technique that is extensively applied to analyze complex software systems [17, 67, 86], causality analysis is proficient at disentangling the complex relations between various factors into an intuitive causal graph with high interpretability. Typically, it comprises two phases: causal discovery and subsequent analysis.

Causal Discovery. Causal discovery seeks to infer causal relations between variables from observational data. These inferred causal relations are often represented as a directed acyclic graph (DAG), referred to as Causal Graph. Formally,

Definition 1 (Causal Graph). *A causal graph is a DAG with nodes V and edges E, i.e., $G = (V, E)$, where each node X ($X \in V$) represents a random variable and each edge $X \rightarrow Y$ ($X, Y \in V$) represents a directed causal relation from X to Y. $Pa_G(X)$ denotes the parent nodes of X in G, i.e.,*

$Pa_G(X) = \{N | N \rightarrow X \in E\}$. *A node Z is the* ***confounder*** *of a given pair of nodes X and Y if it is the common parent of both X and Y, i.e., $Z \in Pa_G(X) \cap Pa_G(Y)$.*

Definition 2 (Endogenous and Exogenous Nodes)**.** *Nodes in the causal graph can be categorized into two distinct groups based on the presence or absence of their parent nodes:* ***endogenous*** *nodes, whose values are determined by other nodes within the graph (i.e., for endogenous node X, its value is determined by the structural equation $X = f_X(Pa_G(X), \epsilon_X)$ where f_X is a deterministic function and ϵ_X is the noise term), and* ***exogenous*** *nodes, which derive their values from external factors (note that the value of exogenous nodes cannot be represented by a structural equation in the same way as that of endogenous nodes due to the absence of external factors in the causal graph).*

With the definition of causal graph, causal discovery is viewed as a process of constructing the DAG, which contains the qualitative structure of the causal relations between variables, from observational data. In this paper, we treat the style of prompt, linguistic features of the prompt, and code metrics of the generated code as nodes of a complicated causal graph. By adopting the state-of-the-art gradient-based causal discovery algorithm DiBS [45] to construct the causal graph, we aim to unveil the causal relations between the prompt and the generated code. Sec. 4 will detail the causal graph construction.

Analysis based on Causal Graphs. Further analysis, including qualitative and quantitative analysis, can be performed once the causal graph has been obtained. Qualitative analysis typically leverages structural information contained within the causal graph. By traversing the graph, all factors that have a causal effect on the target variable can be identified. Furthermore, quantitative analysis, also known as *causal inference*, aims to measure the causal effect of one variable X on another variable Y, by answering *counterfactual* questions such as "what would happen to Y if X were set to a different value?". Such counterfactual value of Y provides insights into the causal effect of X on Y in a quantitative manner, facilitating the interpretation of the causal relations between variables. In practical analysis, users are capable of training a model (see DML in Sec. 4.3 for details) from observational data with the causal graph as a guide, to precisely estimate the causal effect from one variable to another. In this paper, both qualitative and quantitative analyses are performed on the causal graph for different purposes, which will be detailed in Sec. 4. In our context, this framework enables us to understand specific prompt features that influence an attribute of the generated code, and to provide actionable insights to improve the quality accordingly.

3 Research Motivation

In this section, we present the motivation behind our research and the research questions we aim to address. We first provide a motivating example to qualitatively illustrate the importance of prompt in code generation. Then, we conduct a pilot study to quantitatively demonstrate the effectiveness of prompt adjustment through rephrasing. Finally, we discuss the necessity of causality analysis in understanding the complex relations in LLM-based code generation.

3.1 Effect of Prompt in Code Generation

As reviewed in Sec. 2.1, while recent work has shown promising results in prompt engineering, the design of prompts is still largely a manual process and relies on human expertise. Thus, evaluating the quality of prompts remains an open problem [8, 73], let alone providing a systematic guideline for prompt design. Overall, considering the high flexibility of prompts in natural language, we see that the effects of different types of prompts on LLM performance are not well understood. Specifically, the criteria for assessing prompt quality are unclear, making it challenging to determine whether one prompt is superior to another before actually executing it. Moreover, the effects of

prompts are often complicated [29], making it difficult to predict the performance of a prompt solely based on its design.

Fig. 1 presents an example of how the performance of a code generation model can be affected by subtle modifications in the prompt design, leading to highly confusing and non-monotonic results. In this case, the LLM is instructed to generate a program that returns the first letter of each word in a given string. A slightly ambiguous prompt in Fig. 1(a) misleads the LLM into generating a program that capitalizes the initial letter of each word. In contrast, the more explicit instructions provided by the prompt in Fig. 1(b) direct the LLM to generate the correct program. Evidently, this distinction was not anticipated by the designer, as the generated code for two semantically equivalent prompts should be identical. This example illustrates the importance of comprehending the mechanism of prompts' effect on code generation. Consequently, this work aims to answer the following research question:

How to systematically establish the prompt's influence on LLM-based code generation?

3.2 Prompt Adjustment via Rephrasing

Following the observation in Fig. 1, we interpret that "prompt adjustment" is critical, as a minor change in the prompt may substantially improve the performance of a given LLM from generating incorrect code (Fig. 1(a)) to correct code (Fig. 1(b)). Thus, the intuition is that by properly adjusting the prompts, we expect to observe notable changes in various characteristics of the generated code. Then, we can comprehensively establish the effect of the prompt on code generation. An ideal prompt adjustment method shall notably alter one or more properties of the prompt while preserving its semantics. This adjustment, however, is challenging [58, 84, 89], as conventional techniques like synonym substitution can only make trivial changes by replacing words with their synonyms, without altering high-level properties like the prompt's style.

Table 1. Pilot study on the impact of rephrasing.

Instruction	Pass Rate	Syntax Error Rate	Stability
Short	-33.84%	+67.50%	+11.65%
Long	+110.84%	-9.75%	-4.01%
Formal	+71.17%	-50.25%	+0.24%
w/o	0.00%	+19.50%	+6.33%

In this work, we propose adjusting the prompt through *rephrasing*, which has embodied the characteristics of an ideal prompt adjustment method as shown in Fig. 1. Specifically, we collect 100 prompts where each associated with a programming problem, and measure the code metrics on them under three instructions accordingly, with "w/o" denoting the control group where the LLM is prompted to rephrase without any rephrasing intention. By explicitly asking an LLM to rephrase the prompt, often depicting a programming problem, in different manners, we can systematically and smoothly modify the style of the prompt while keeping its semantics.[1] Moreover, the results of a small-scale pilot study, as shown in Table 1, indicate that rephrasing can effectively affect the generated code. In this pilot study, we ask ChatGPT [5] to rephrase the given programming problem by following three simple instructions: make the prompt shorter, longer, or more formal. As a baseline, we also set up a control group that receives no instructions. Then, all rephrased programming problems are fed into LLM to generate code, which is then compared to the code generated from the original prompt. In this table, the pass rate is the percentage of generated code that passes the test cases, the syntax error rate is the percentage of generated code that contains syntax errors, and the stability is the magnitude measuring the similarity between codes generated from the same prompt. The formal definitions of pass rate and syntax error rate are provided in Sec. 5.2. For stability, it is equivalent to `mut_sim_B`. We use these metrics (e.g., pass rate) to offer

[1]Other factors, such as the type of programming problem, are out of the scope of this study. We use the aggregated results to mitigate the impact of these factors.

initial insights, and our main study in Sec. 7 supplements them with finer-grained assessments to better understand the code generation quality. The results show that rephrasing instructions can indeed affect the generated code and, more importantly, convey different intentions to the LLM, which are reflected in the varied change trends of code metrics.[2] As will be shown in Sec. 6, we validate the effectiveness of rephrasing in a more comprehensive pilot study. Furthermore, the selection of rephrasing instructions offers a systematic way to explore the space of prompts, which is otherwise intractable. Thus, this work adopts LLM-based rephrasing as the basic operator and aims to answer the following question:

> How to systematically adjust the prompt and effectively explore its space to achieve the desired effect on the generated code?

3.3 Analysis for Relations in Code Generation

A straightforward approach to analyzing the relation between multiple variables is to identify and calculate the correlation between them. We argue, however, that correlation is insufficient for analyzing complex relations like those in LLM-based code generation.

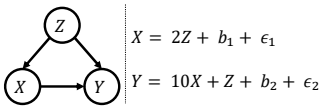


Fig. 2. Illustration of causality.

Fig. 2 illustrates the difference between causality and correlation. Here, X represents the prompt's quality, Y represents the generated code's quality, and Z represents one property of the prompt, such as its difficulty level. Suppose, for instance, these three variables follow the quantitative relations presented on the right side of Fig. 2. X's value is determined by Z, and Y's value is determined by both X and Z. In this case, Z is a *confounder* (refer to Sec. 2.3) of the relation between X and Y. Besides, b_1 and b_2 are two constants, and ϵ_1 and ϵ_2 denote random noise with zero mean.

Causality vs. Correlation. The standard procedure for predicting the value of Y based on the correlation is to train a regression model taking X and Z as inputs. This trained model is capable of achieving a high level of accuracy, but it may not capture the correct quantitative relation between X and Y. For example, $Y = 21Z + 10b_1 + b_2$ is a "perfect" model to offer 100% accuracy in estimating.[3] However, it is evidently not the appropriate model for examining the effect of the prompt (X) on the generated code (Y). Any downstream analysis based on this model will inevitably lead to an incorrect conclusion that X has no effect on Y. This problem will be further exacerbated in real-world scenarios with hundreds of variables, such as the LLM-based code generation task. On the contrary, causality analysis first identify the confounder Z and try to remove its effect, then measure the direct effect of X on Y (see Sec. 4.3 for details), thereby revealing the correct quantitative relation between X and Y. This unbiased estimation achieved by causality analysis, we believe, is essential for understanding the complex relations in LLM-based code generation.

Furthermore, the large number of variables involved in LLM-based code generation, the interactions between multiple confounders, the intricacy of their joint effects can make it challenging to pinpoint the exact influence of each variable. This complexity adds significant uncertainty to the interpretation of the causal relationships and highlights the importance of careful model specification and sensitivity analysis in real-world applications. Additionally, the uncertainty of the LLM's output further complicates the trial-and-error process of empirical analysis [51, 82]. Specifically, the generation for a given prompt is stochastic rather than deterministic. Distributional estimation with epistemic uncertainty, rather than a singular or a few executions, deserves more

[2]To clarify, we aim to demonstrate that rephrasing can affect the generated code, as reflected in the change trends of these metrics. "Rephrasing" however does **not** necessarily mean "improving" the generation.

[3]All noise terms are omitted here because their mean is zero.

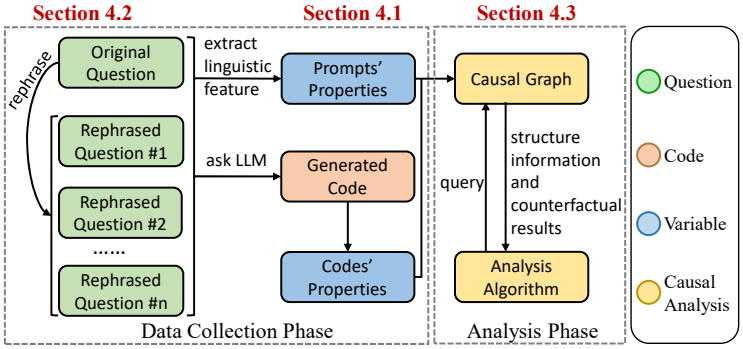


Fig. 3. Study overview.

attention in assessing LLMs' performance, which is the key focus of causality analysis [54]. To conclude, we attempt to address the following issue in this paper:

> How to disentangle the intricate relations between numerous variables in a nondeterministic system — LLM-based code generation — and to identify the correct quantitative relations between prompts and the generated code?

4 Design

We provide an overview of our study in Fig. 3 and describe their inter-connections as follows. In our study, two primary phases are involved: *data collection* and *analysis*, which is structured to systematically investigate the prompt's influence on LLM-based code generation (RQ1), explore prompt adjustment strategies (RQ2), and disentangle complex variable relations (RQ3). In the first phase, we begin by rephrasing prompts, i.e., programming questions (the green blocks in Fig. 3). Then, both rephrased and original prompts are fed into two subtasks. The first subtask is to quantify the characteristics of prompts, which is accomplished by extracting linguistic features from the natural language text in these questions. Simultaneously, questions are fed into an LLM to generate code, with a variety of code performance metrics like correctness and readability being collected. Together, the linguistic features and the code performance metrics (blue blocks in Fig. 3; see detailed setup in Sec. 5.2 and Sec. 7.1) constitute the data for the analysis phase. During the analysis phase, we first construct a causal graph that represents the causal relations between all the variables that we collected. A series of further analyses are conducted on the causal graph in an effort to identify general principles for code generation prompt design. Here, each causal relationship represents the quantitative mechanism of how one variable affects another.

Overall, the workflow in Fig. 3 is designed to address the research questions and subsequent technical challenges raised in Sec. 3. Specifically, Sec. 4.1 proposes a systematic approach for quantifying the characteristics of prompts. Then, Sec. 4.2 addresses the challenge of prompt adjustment raised in Sec. 3.2. Afterwards, Sec. 4.3 adopt causality analysis to disentangle the complex relations between the numerous variables involved in this study. Finally, Sec. 4.4 proposes an algorithm for learning and explaining prompts' effect on code generation, addressing the issue raised in Sec. 3.3.

4.1 Prompt Quantification

To systematically establish the prompt's influence on LLM-based code generation (RQ1), as noted in Sec. 3.1, we must first address the unclear criteria for assessing prompt quality, which currently hinders pre-execution comparisons. We propose quantifying prompt characteristics using linguistic features, providing a measurable foundation to understand their impact on code generation. Linguistic features, as the structural elements of natural language, such as morphology, syntax,

and semantics, enable us to capture prompt variations systematically. We follow Lee et al. [37, 38], a state-of-the-art study on linguistic features in NLP, to extract linguistic features from prompts. Specifically, a total of 255 linguistic features are extracted from prompts, including *lexical* (e.g., count of nouns), *syntactic* (e.g., phrasal features [47]), and *semantic* (e.g., semantic richness [37]) features. The full list of linguistic features can be found at the LFTK website [4].

Holistically, this work models the adjustment of LLM-based code generation by adhering to the following causal chain: meta-prompt (rephrasing prompt) → prompt (programming question) → code metrics, where arrows represent the impact propagation. In practice, since textural prompts are unmeasurable, we concretize them by linguistic features extracted from the prompts. Then, the causal chain is transformed into meta-prompt → linguistic features → code metrics. For instance, applying distinct meta-prompts (e.g., "short" versus "long" instructions) directly influences linguistic attributes like total word count (t_word), thereby altering functional code properties such as test case pass rate (pass_rate). This illustrates how causal relationships between prompt and code quality can be captured by these quantitative metrics and features.

We believe these linguistic features are sufficient to surrogate all interactions between prompts and other variables. The rationale behind this assumption is that rephrasing intentions are designed to change the syntactical property of the prompt while preserving its semantics. Therefore, these linguistic features can effectively capture the changes in the prompt as the semantics of the prompts are unchanged after the rephrasing. This transformation is crucial for the subsequent causal analysis, as it enables us to focus on structured and quantifiable variables rather than unstructured and unmeasurable text. As will be further validated by our empirical results (Table 5), these linguistic features demonstrate strong predictive power in capturing prompt effects on code metrics.

4.2 Rephrase Generation

To systematically adjust prompts and effectively explore their space for desired effects on generated code (RQ2; see Sec. 3.2), we propose using LLMs to rephrase prompts. Rephrasing can generate a greater variety of prompts and substantially improve the prompt diversity, indicating that the value space for each linguistic feature has been thoroughly explored. Hence, we presume that the use of rephrasing is beneficial to the observability of the linguistic features, which is crucial to the success of subsequent analysis.

Inspired by mature rephrasing tools like QuillBot [6], we designed a prompt template for programming question rephrasing, as shown in Fig. 4. This template also includes a set of rephrasing intentions that can direct the LLM to rephrase the given prompt in a particular manner. These intentions are classified as instruction, role, and scenario. *Instruction* provides the LLM with explicit instructions, such as *make it short*. For *role* and *scenario*, they are designed to provide context for the LLM, like *as a student* and *in a programming competition* to unleash the LLM's creativity. Furthermore, pre-defined intentions from various categories can be combined to form a more complex rephrasing intention. These rephrasing intentions are intended to exhaustively cover applicable rephrasing preferences for the prompt rephrasing task. Once a combination of *instruction*, *role*, and *scenario* is selected (e.g., *make it short* + *as a student* + *in a programming competition*), we can generate a rephrased prompt by filling in the template in Fig. 4(a) with the user-provided programming question.

We designed over 20 rephrasing intentions, which can be combined to form over one hundred filled-in content for the meta-prompt template. To avoid ambiguity, we use *meta-prompt* to refer to the prompt used for rephrasing programming questions and *prompt* to refer to the programming question itself. All synthesized meta-prompts are refined and optimized using the state-of-the-art

[4] https://lftk.readthedocs.io/en/latest/index.html

(a) prompt template for programming question rephrasing

User:
Role + Scenario. Please rephrase the programming problem in XML tags while keeping its original meaning and structure, to help enhance the understanding of the problem's intent and facilitate better code generation. Instruction. You should keep all mathematical symbols in latex format. Here's the original problem: \n<text>\nQuestion \n</text>

(b) rephrase intention

	Name	Filled-in Sentence
Instruction	Long	Expand the problem in a more detailed and thorough manner. Make the explanation longer and clearer.
	Short	Condense the problem in a more concise and clear manner. Make the explanation shorter while maintaining clarity.
	Formal	Rewrite the problem in a more formal and clear manner.
	Fluent	Rewrite the problem in a more fluent and clear manner.
	Technical	Rewrite the problem in a more technical and detailed manner.
	Logical	Rewrite the problem to make it more logical and clear.
Role	Student	You are a student majoring in software engineering.
	Dev	You are a senior python programmer.
	Comp	You are a competitor in a programming competition.
Scenario	Clearer	The following programming question is not clear enough, so you need to rephrase it.
	Improve	Someone wrote the following programming question, and asked you to improve it to make it clearer.
	Specify	You need to rephrase programming questions to make them more suitable for python3.

Fig. 4. Meta-prompt design. Red text indicates the programming question filled in by the user, and blue text indicates the pre-defined rephrasing intention to be selected.

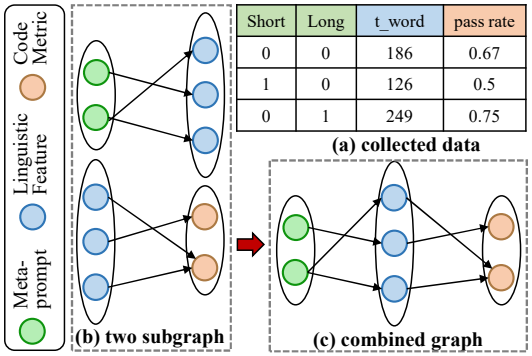
LLM, GPT-4o, to enhance the quality of the rephrased prompts, thereby improving the quality of the generated code. This approach aligns with best practices in state-of-the-art LLM-based prompt optimization studies [84]. In this paper, we focus on the 12 rephrasing intentions listed in Fig. 4(b), considering both expert experience and computational cost. It is worth noting that the design of rephrasing intention is not limited to those shown in Fig. 4(b). Additional rephrasing intention can be added to the template to expand the diversity of the prompts, and the subsequent analysis can be conducted in the same manner.

4.3 Causality Analysis

To comprehensively understand the intricate relations between variables involved in this study (RQ in Sec. 3.3), we employ causality analysis, which typically consists of two steps: causal discovery (construct the causal graph) and causal inference (subsequent analysis on the graph).

Causal Discovery. To disentangle the intricate relations among numerous variables in LLM-based code generation and identify their correct quantitative relationships (RQ3), as raised in Sec. 3.3, our study analyzes three variable groups: meta-prompt variables (0/1 variables flagging whether users select a particular rephrasing intention listed in Fig. 4(b) or not), linguistic feature variables, and code metric variables. By employing causal discovery, it is possible to disentangle the complex relations between variables and learn a causal graph, where the nodes represent the variables and the edges represent the causal relations between nodes. In this study, we employ a state-of-the-art causal discovery algorithm, DiBS [45]. As a gradient-based causal discovery algorithm, DiBS transforms the causal graph construction procedure into a differentiable optimization problem, thereby substantially enhancing the efficacy of causal discovery and ensuring the validity of the learned graph [45]. With DiBS, we leverage its standard assumptions, such as causal sufficiency, to

learn an accurate causal graph. This aligns with practices commonly adopted in other software engineering research [22].



Short	Long	t_word	pass rate
0	0	186	0.67
1	0	126	0.5
0	1	249	0.75

(a) collected data

Fig. 5. Two-step causal discovery.

However, the large number of variables involved in this study makes learning causal graphs exceedingly difficult. To address this issue, we propose a two-step causal discovery approach, which is shown in Fig. 5. As discussed in Sec. 4.1, the adjustment of LLM-based code generation can be modeled as follows: meta-prompt → linguistic features → code. By assuming that the linguistic features are sufficient to work as a proxy for the meta-prompt characteristics, we divide the whole causal graph that contains meta-prompt, linguistic features, and code into two subgraphs. Specifically, the first subgraph contains the relations between meta-prompt and linguistic features, and the second subgraph contains the relations between linguistic features and code metrics. It is worth noting that this assumption does not imply that variables in the same group will not have a causal relationship. Instead, this assumption merely limits the possible causal relationships between variables in different groups. After learning the two subgraphs independently, the complete graph can be obtained by concatenating them directly.

Causal Inference. After learning the causal graph, we conduct quantitative analysis on the graph to identify the causal effect of one variable on another. Taking Fig. 2 as an example, the quantitative analysis seeks to know the magnitude of the change in Y (the *output* variable) when X (the *treatment* variable) changes from x_1 to x_2 while blocking the effect of *confounding* variable Z. Here, x_1 and x_2 are two specific values of X that are determined by the user of causal inference and may not be observed in the data. Therefore, they are counterfactual values, denoted as $do(X = x_1)$ and $do(X = x_2)$, respectively. Y's average change reflects the causal effect of X on Y. This is denoted as the *average treatment effect* (ATE) within the context of causality analysis [49, 54]. To avoid ambiguity, we denote the ATE of X on Y as ATE_X^Y in the following sections. Below, we present the formal definition of ATE:

Definition 3 (ATE). *ATE of treatment variable X on output variable Y is defined as:*

$$\text{ATE}_X^Y = \mathbb{E}[Y \mid do(X = \boldsymbol{x}_1)] - \mathbb{E}[Y \mid do(X = \boldsymbol{x}_2)] \tag{1}$$

We employ double machine learning (DML) [13] to estimate the value in Eq. 1. DML is a state-of-the-art method and has been extensively applied in the causality literature [4]. It first uses two machine learning models to estimate treatment variables and output variables, respectively, from confounders. The causal effect of the treatment variables on the output variables can then be determined by training another machine learning model that predicts the relations between the residuals of the two models. In this study, we employ the DML implemented in EconML [4]. To understand it, we provide an example that perform DML on the causal graph in Fig. 2.

Example 4.1. The steps of DML on the causal graph in Fig. 2 are as follows:

Step 1. Estimate the value of X based on Z in the form of $X = \alpha \cdot Z$. Here, $\alpha = 2$ and $X - \hat{X} = b_1$.

Step 2. Estimate the value of Y based on Z in the form of $Y = \beta \cdot Z$. Since X is determined by Z, we have $\beta = 21$ and $Y - \hat{Y} = 10b_1 + b_2$.

Step 3. Estimate the value of $Y - \hat{Y}$ based on $X - \hat{X}$ in the form of $Y - \hat{Y} = \gamma(X - \hat{X})$, i.e., $10b_1 + b_2 = \gamma \cdot b_1$. Therefore, we have $\gamma = 10$, aligning with the true causal effect of X on Y.

4.4 Establishing Prompt Effect on Code

Algorithm 1: Analysis

Input: Causal Graph G, Meta-prompt Variable M, Linguistic Features $L_l = (L_1, \cdots, L_n)$, Code Metrics $C_l = (C_1, \cdots, C_n)$
Output: Dictionary D of Influenced Code Metrics with Explanation for the Effect

1 $D \leftarrow \{\}$;
2 **foreach** $C \in C_l$ **do**
3 $\quad \text{ATE}_M^C \leftarrow \mathbb{E}[C \mid do(M = 1)] - \mathbb{E}[C \mid do(M = 0)]$;
4 **end**
/* Find the top 3 code metrics that are most affected by M */
5 Sort C_l in descending order according to $\texttt{abs}(\text{ATE}_M^C)$;
6 $C_S \leftarrow \texttt{Top3Affected}(C_l)$;
7 **foreach** $C \in C_S$ **do**
8 $\quad L_P \leftarrow \texttt{IdentifyAncestors}(G, C, L_l)$;
9 $\quad$ **foreach** $L \in L_P$ **do**
10 $\quad\quad l_{M=0} \leftarrow \mathbb{E}[L|M = 0]$; $l_{M=1} \leftarrow \mathbb{E}[L|M = 1]$;
11 $\quad\quad \text{ATE}_L^C \leftarrow \mathbb{E}[C \mid do(L = l_{M=1})] - \mathbb{E}[C \mid do(L = l_{M=0})]$;
12 $\quad$ **end**
$\quad$ /* Find the linguistic features that are mainly responsible for the effect of M on C */
13 $\quad D[C] \leftarrow \texttt{SortSelect}(L_P)$;
14 **end**
15 **return** D

Alg. 1 presents the prompt effect analysis procedure, finalizing our approach to systematically establish the prompt's influence (RQ1) and identify quantitative relations in LLM-based code generation (RQ3). It takes as inputs a causal graph G (from Sec. 4.3), a meta-prompt variable M, linguistic features L_l (from Sec. 4.1), and code metrics C_l. Here, M is typically a binary variable, indicating whether a specific rephrasing intention (listed in Sec. 4.2) is selected ("1" denotes selection), enables a comprehensive analysis of prompt effects.

Holistically, Alg. 1 comprises two steps, **learning** and **explaining** the effect. In the first step, we compute the average treatment effect (ATE) of the meta-prompt variable M on each code metric C (lines 2-4) and then select the top three code metrics that are most affected by M (lines 5-6). This step is intended to quantify the extent of M's effect on code generation. In the second step, we attempt to explain how the changes in meta-prompt M are propagated to the selected code metrics C_S (lines 7-13). Specifically, we first query the causal graph to find all the ancestor linguistic features of the chosen code metric C (line 8), as they are potential mediators of the effect of M on C. Then, we intervene on each ancestor individually to calculate their ATE on C (lines 10-12). Based on the calculated ATE, we determine the linguistic features primarily responsible for the effect of M on C (lines 13). Finally, we return a dictionary D containing the top three influenced code metrics with explanations (line 15).

5 Experiment Setup

The scripts required to conduct the study are written in Python3 with about 2.4K lines of code. We run all experiments on a server with AMD Ryzen 3970X CPU and one NVIDIA RTX 3090 GPU.

5.1 Datasets & Model

Datasets. This paper focuses on one of the most popular programming languages, Python3. Hence, we select APPS [25] and BigCodeBench (BCB for short) [90]. APPS is a dataset exclusively designed for Python code generation, containing 10K Python problems with difficulty annotations. Each problem is associated with a few correct solutions and a set of test cases. The difficulty levels of problems in APPS exhibit sufficient diversity, ranging from easy ("Introductory") to hard

Table 2. Code metrics employed in this study

Category	Name	Description	Justification
Correctness (5)	pass_rate	The pass rate of test cases.	Measured via automated testing frameworks with the testcases from the benchmark
	run_err_rate	The runtime error rate of test cases.	Derived from execution logs, ensuring consistent error detection.
	syn_err	The number of syntax errors revealed by *tree_sitter* [3].	*tree-sitter* [3] is a popular parser used in IDEs like VS Code.
	gold_sim_CB	The similarity (in CodeBLEU [59]) between the generated and ground truth.	CodeBLEU [59] is a specialized metric for code, validated in prior studies.
	gold_sim_B	The similarity (in BLEU [52]) between the generated and ground truth.	BLEU [52] is a standard metric with proven reliability in general NLP tasks.
Diversity (2)	mut_sim_CB	The mutual similarity (in CodeBLEU) among the generated solutions.	Same as gold_sim_CB; ensures consistent diversity measurement.
	mut_sim_B	The mutual similarity (in BLEU) among the generated solutions.	Same as gold_sim_B; widely accepted for diversity measurement.
Overhead (1)	timeout_rate	The timeout rate of test cases.	Measured via execution time limits, as a standard practice in testing.
Readability (1)	black_count	The number of places reported by *black* [1] where PEP8 is violated.	*black* [1] is a widely-adopted PEP8 formatter, ensuring reliable readability checks.
Security (1)	semgrep_count	The number of potential security bugs revealed by *Semgrep* [2].	*Semgrep* [2] is an industry-standard static analysis tool tailored for security auditing.

("Competition"), which is beneficial for our study. BigCodeBench is another mainstream LLM code generation dataset [33], which contains 1K Python3 problems with test cases and correct solutions. By introducing third-party libraries and covering a wide range of real-world programming scenarios, BigCodeBench is more challenging and complex than other datasets, such as HumanEval [12] and MBPP [9]. We believe that the combination of APPS and BigCodeBench can provide a comprehensive evaluation of the code generation capability of LLMs. To learn causal graphs, we use all data from BigCodeBench and sample 6K data points from APPS considering the high cost of using LLMs.[5]

Models. Four LLMs are used: GPT-Neo [25], GPT-3.5-Turbo (ChatGPT; GTP-3.5 in short), GPT-4, and Qwen-2.5-Coder (14B; Qwen-2.5 in short) [27, 83]. These four models form a spectrum of LLMs in terms of size and performance, which represent the relatively tiny but powerful LLM, the most widely-used LLM, and the state-of-the-art LLM (both commercial and open-source), respectively. For GPT-3.5 and GPT-4, we use the official API supported by Azure. Except for the maximum length of generated solutions, which is set to 2,000 to handle long solutions, all parameters are set to their default values. To avoid the randomness of LLMs, for each data point, we collect three solutions from each model. In this study, we employ GPT-3.5-Turbo to rephrase the prompts for two reasons. First, compared to code generation, prompt rephrasing is relatively simple, and GPT-3.5 is sufficient (as will be validated in Sec. 6.1). Our preliminary observation shows that the powerful but expensive GPT-4 is an "overkill", while the tiny GPT-Neo cannot handle it plausibly. Second, in this study, we focus on the code generation and the rephrasing itself is quite standard and not our primary focus.

5.2 Code Metrics

Code generation is naturally multi-faceted. For example, generated code may be correct but fail to adhere to recommended coding styles (e.g., PEP8 for Python). To comprehensively evaluate the code generation capability of LLMs, we employ a set of code metrics to measure the quality of generated code. These metrics are categorized into five categories: correctness, diversity, overhead, readability, and security, as reported in Table 2. The rationale behind the selection and categorization is that these metrics are representative and can be used to evaluate the code generation capability of LLMs from different perspectives. We provide further justifications for the reliability of the tools/criteria used to derive these metrics in the table. It is noteworthy that the use of various metrics is intended to demonstrate our framework's capacity to systematically analyze the intricate relations between

[5]Indeed, 6K points are sufficient to learn graphs with reasonable quantity and acceptable cost, though more can be sampled.

prompts and generated code, not implying that our framework is designed to resolve all these code-related issues. Some of these metrics may share similar purposes, but are not redundant. For example, `gold_sim_CB` and `gold_sim_B` measure the similarity between the generated code and the ground truth, but they are based on different similarity scores, CodeBLEU and BLEU, respectively. Including both metrics allows us to evaluate the generated code from different perspectives and provide more insights. We implement these metrics on the basis of [40, 46, 57, 76].

6 Pilot Study

Here, we conduct two pilot studies to validate our approach's fundamental assumptions as follows:

PS1: Effectiveness of LLM Rephrasing. To validate whether the rephrasing of the prompt made by LLMs induces evident changes in the meta-prompt (programming questions).

PS2: Impact of Rephrasing Meta-prompts on Generated Code. To validate whether rephrasing meta-prompts are capable of leading to notable changes in the generated code.

6.1 PS1: Effectiveness of LLM Rephrasing

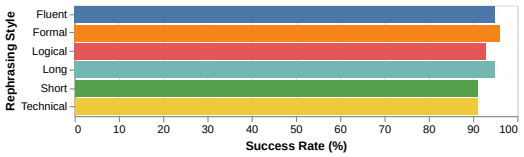


Fig. 6. Effectiveness of LLM rephrasing.

As a crucial premise of our approach, the rephrasing of the programming questions by LLMs is expected to be effective. Therefore, before proceeding to the main study, we first validate the effectiveness of LLM rephrasing. Specifically, we aim to verify whether the rephrased questions generated by the LLM are in line with the rephrasing instructions provided by the meta-prompts. Specifically, for simple instructions (e.g., Long/Short rephrasing), we directly compare the length of the rephrased questions with the original ones. For other instructions, we follow the LLM-as-judge approach [14] to assess the effectiveness of LLM rephrasing (i.e., an external LLM performs the judgments without knowing the ground truth). We randomly sample 100 programming questions from APPS and BigCodeBench, and compare the rephrased questions generated by the LLMs with the original questions.[6] We report the result in Fig. 6. As expected, the success rate of the LLMs in rephrasing the questions as instructed by the meta-prompts is high (93.5% on average). Therefore, we conclude that the rephrasing of the programming questions by LLMs is effective and the subsequent analysis is meaningful.

6.2 PS2: Impact of Rephrasing Meta-prompts on Generated Code

Since this work strives to investigate the impact of prompt rephrasing on the generated code, it is essential to validate whether rephrasing is capable of inducing notable changes in the generated code. To understand the difference in the distribution of code metrics, we leverage the Pillai's Trace test [56], which is a test statistic used in multivariate analysis of variance to determine whether there is a significant difference between the distributions of multiple variables across groups. Similar to PS1, we randomly sample 100 programming questions from APPS and BigCodeBench, and compare the distributions of code metrics between the original questions and the rephrased questions.

Table 3. Pillai's Trace results of different models.

Model	GPT-Neo	GPT-3.5	GPT-4	Qwen-2.5
Pillai's Trace	0.9961	0.9965	0.9986	0.9978
p-value	1.1e-16	1.1e-16	1.1e-16	1.1e-16

We report the result in Table 3. Here, we observe that the p-values are all significantly smaller than the significance level (0.05), indicating that the distributions of code metrics for the original questions and the rephrased questions are significantly different. In the meantime, the

[6]We primarily consider the instructions listed in Fig. 4(b), as "Role" and "Scenario" are designed to aid the LLM in understanding the context of the programming question, rather than inducing a major change in the question.

Pillai's Trace values are close to 1, further confirming the effectiveness of prompt rephrasing in inducing notable changes in the generated code.

7 Evaluation

In this section, we evaluate the efficacy and value of our analysis framework, as described in Sec. 4. We first evaluate the accuracy of the learned causal graphs, which serve as the foundation of our framework (**RQ1**). Then, by querying the learned causal graphs, we obtain several valuable and in-depth insights into LLM-based code generation (**RQ2**).

7.1 RQ1: Verification of the Causal Graphs

The lack of a ground-truth graph constitute one of the major limitations of causal discovery algorithms. To alleviate this issue, we gauge the accuracy of the learned graphs by two approaches: human evaluation and predictive capability evaluation.

Table 4 reports the statistics of the learned causal graphs and the result of human evaluation [7]. Since the causal graphs for distinct models can vary significantly [10], we launch the causal discovery algorithm separately for each model. For each model, we first collect values for 12 meta-prompt variables (see Sec. 4.2), 255 linguistic variables (see Sec. 4.1), and 10 code metrics (see Sec. 2). The overwhelming number of linguistic variables is then reduced by removing non-correlated variables, as correlation is typically a necessary condition for causation. After causal discovery, nodes with no edges are also removed for the same reason.

Human Evaluation. We first conduct a human evaluation to assess the accuracy of the learned causal graphs. Aligning with the previous study [31], we sample two subgraphs with 15 to 20 nodes from each learned causal graph and ask five experts (Ph.D. or Ph.D. candidates with expertise in LLM and code generation) to evaluate the learned edges. It is worth noting that the subgraph sampling is intended to reduce the evaluation complexity for individual experts, as the full graph is too large to be evaluated in a reasonable amount of time. However, each edges in the full graph is considered and evaluated in the assessment. Before the evaluation, each expert is educated on the definition of the causal graph and the meaning of the nodes. We presume that the experts are capable of determining the causal relations depicted in the graph based on their domain knowledge and experience (e.g., whether the "longer" rephrasing instruction causes a change in the total word count of the rephrased prompt, or whether the length of the prompt would have a positive effect on the correctness of the generated code). After the education, experts are requested to mark edges that they believe are incorrect and edges that they believe are missing based on their domain knowledge. Correspondingly, we employ two metrics to evaluate the accuracy of the learned graphs: error rate (the portion of incorrect edges in the learned graph) and negative predictive value (NPV; the ratio of the number of reported missing edges to the total number of edges in the learned graph).

Table 4. Statistics of the learned causal graphs.

	models	nodes	edges	error rate	NPV	Kappa
APPS	GPT-Neo	45	192	3.19%	2.12%	0.87
	GPT-3.5	50	177	9.26%	4.26%	0.80
	GPT-4	45	180	2.75%	2.86%	0.82
	Qwen-2.5	46	224	4.15%	3.48%	0.81
BCB	GPT-Neo	44	158	1.98%	1.40%	0.94
	GPT-3.5	45	134	2.05%	1.98%	0.92
	GPT-4	44	190	3.33%	2.26%	0.86
	Qwen-2.5	46	211	2.74%	1.23%	0.89

As shown in Table 4, the error rate and NPV values are low, indicating that the learned graphs are accurate and reliable. In particular, both the error rate and NPV are below 10% for all models. The accuracy of the learned graph for GPT-3.5 is slightly lower than other models on APPS. We attribute this to higher node numbers, which directly lead to higher complexity, increasing the difficulty of causal discovery. We also present Fleiss' Kappa [20], a statistical measure of inter-rater agreement (the higher the value,

[7] An example of the causal graph can be found at https://anonymous.4open.science/r/CALL-E7F2/graphs/neo.svg

Table 5. Predictive capability of the learned causal graphs.

	GPT-Neo		GPT-3.5		GPT-4		Qwen-2.5	
	APPS	BCB	APPS	BCB	APPS	BCB	APPS	BCB
pass_rate	0.8431	0.7963	0.7600	0.7713	0.8206	0.8192	0.8315	0.8226
run_err_rate	0.8492	0.8022	0.7978	0.7920	0.8474	0.8334	0.8460	0.8416
syn_err	0.7196	0.7274	0.7812	0.7739	0.8363	0.8537	0.8411	0.8443
gold_sim_CB	0.9221	0.8395	0.8432	0.8169	0.8708	0.8048	0.8759	0.8253
gold_sim_B	0.9026	0.8269	0.8657	0.8317	0.8785	0.8185	0.8874	0.8481
mut_sim_CB	0.9068	0.9281	0.8654	0.8638	0.8878	0.8923	0.8836	0.8779
mut_sim_B	0.9120	0.9193	0.8805	0.8728	0.8928	0.8883	0.8772	0.8734
timeout_rate	0.6157	0.7233	0.7101	0.7066	0.6686	0.6980	0.6878	0.7370
black_count	0.7854	0.7852	0.8099	0.8161	0.8638	0.8616	0.8538	0.8519
semgrep_count	1.0000	1.0000	0.1812	0.1394	0.1448	0.1267	0.1375	0.1484

the better), to evaluate the consistency of the human evaluation. The Kappa values are all above 0.8, indicating a high level of agreement among the experts. Therefore, we do not take specific actions to address the disagreement.

Predictive Capability Evaluation. To further evaluate the accuracy of the learned causal graphs, we propose an efficient verification strategy that uses causal inference methods, which are based on the learned graph, to predict each downstream variable (code metrics listed in Table 2) and then compares the predictions to the actual values. Ideally, the learned graph would depict the true causal relations between each pair of variables, thereby precisely depicting the propagation of changes from meta-prompt variables to code metrics. Consequently, the predictive potential of the learned graph should correctly reflect its accuracy. Here, we report the R^2 score to quantify the predictive capability of the learned causal graphs. The R^2 score quantifies the proportion of the variance in the target variables that can be predicted from the input variables, with a higher score indicating a better predictive capability.

Table 5 reports the verification results. In general, the learned graphs exhibit a strong predictive capability, as indicated by the high R^2 scores, which are typically above 0.8 in most cases. The only exception is the `semgrep_count` metric, which represents the number of security issues detected by Semgrep [2] in the generated code. On this metric, the graph for GPT-Neo performs perfectly, with an R^2 score of 1.0, while other models have a poor performance. We investigate this phenomenon and discover that the `semgrep_count` has a low variance. Specifically, zero security issues are detected for code generated by GPT-Neo, while 13 security issues are detected on GPT-3.5, four on GPT-4, and nine on Qwen-2.5. We presume that GPT-Neo has been fine-tuned to the point of relative overfitting on APPS, a clean dataset, resulting in no detected security flaws. For other models, we attribute the impressive performance to there innovative training paradigm, reinforcement learning from human feedback (RLHF) [91]. In the subsequent analysis, `semgrep_count` will be omitted as no meaningful insights can be obtained from it.

There are several other interesting observations. Across different models, the learned graph of GPT-Neo exhibits a substantially distinct performance pattern compared to the graphs of other models. In particular, this graph performs outstandingly on similarity-based metrics for APPS (e.g., `gold_sim_CodeBLEU`). This phenomenon is interpreted as a consequence of the overfitting of GPT-Neo on APPS. Instead of "thinking" and then writing code like human programmers, we interpret that GPT-Neo is more likely to memorize a large number of tiny code snippets from training data and then combine them according to the requirements. Such characteristics render GPT-Neo insensitive to changes introduced by meta-prompt variables, preserving the stability and similarity of the generated code to the ground truth. In terms of the difference between APPS and BigCodeBench (BCB), we observe an evident performance drop in the correctness-related metrics (see Table 2). This is attributed to the fact that BCB is a more challenging dataset, containing

			GPT-Neo		GPT-3.5-Turbo		GPT-4		Qwen-2.5-Coder	
			APPS	BCB	APPS	BCB	APPS	BCB	APPS	BCB
Instruction	Long	#1	syn_err	gold_sim_B	gold_sim_B	run_err_rate	gold_sim_B	black_count	mut_sim_B	gold_sim_CB
		#2	black_count	run_err_rate	gold_sim_CB	syn_err	mut_sim_B	syn_err	syn_err	pass_rate
		#3	run_err_rate	pass_rate	syn_err	mut_sim_B	syn_err	gold_sim_CB	gold_sim_B	gold_sim_B
	Short	#1	syn_err	run_err_rate	gold_sim_B	gold_sim_CB	pass_rate	run_err_rate	run_err_rate	syn_err
		#2	gold_sim_CB	gold_sim_CB	run_err_rate	syn_err	syn_err	gold_sim_B	gold_sim_B	run_err_rate
		#3	black_count	black_count	syn_err	black_count	black_count	syn_err	black_count	gold_sim_CB
	Formal	#1	gold_sim_B	mut_sim_CB	mut_sim_CB	mut_sim_B	gold_sim_B	gold_sim_CB	mut_sim_CB	black_count
		#2	gold_sim_CB	syn_err	gold_sim_CB	gold_sim_B	mut_sim_B	pass_rate	mut_sim_B	gold_sim_CB
		#3	black_count	mut_sim_B	run_err_rate	pass_rate	mut_sim_CB	mut_sim_B	gold_sim_CB	mut_sim_B
	Fluent	#1	black_count	gold_sim_B	gold_sim_CB	gold_sim_B	gold_sim_CB	syn_err	gold_sim_B	syn_err
		#2	syn_err	pass_rate	gold_sim_B	syn_err	gold_sim_B	mut_sim_B	run_err_rate	gold_sim_B
		#3	gold_sim_B	black_count	timeout_rate	mut_sim_B	run_err_rate	gold_sim_B	gold_sim_CB	mut_sim_B
	Technical	#1	gold_sim_CB	mut_sim_CB	gold_sim_B	gold_sim_B	gold_sim_B	gold_sim_B	gold_sim_CB	black_count
		#2	mut_sim_CB	gold_sim_B	gold_sim_CB	black_count	run_err_rate	gold_sim_CB	mut_sim_B	mut_sim_B
		#3	mut_sim_B	pass_rate	timeout_rate	gold_sim_CB	mut_sim_B	black_count	run_err_rate	gold_sim_B
	Logical	#1	pass_rate	gold_sim_B	gold_sim_B	gold_sim_CB	mut_sim_CB	mut_sim_B	syn_err	mut_sim_CB
		#2	mut_sim_B	run_err_rate	syn_err	mut_sim_B	pass_rate	gold_sim_B	mut_sim_CB	syn_err
		#3	mut_sim_CB	syn_err	gold_sim_CB	run_err_rate	mut_sim_B	syn_err	pass_rate	gold_sim_B
Role	Student	#1	black_count	mut_sim_B	gold_sim_CB	mut_sim_B	gold_sim_B	mut_sim_CB	mut_sim_B	gold_sim_B
		#2	syn_err	gold_sim_B	gold_sim_B	gold_sim_B	mut_sim_B	pass_rate	mut_sim_CB	black_count
		#3	mut_sim_CB	syn_err	timeout_rate	gold_sim_CB	gold_sim_CB	gold_sim_CB	pass_rate	timeout_rate
	Dev	#1	gold_sim_CB	gold_sim_B	run_err_rate	pass_rate	gold_sim_CB	gold_sim_B	gold_sim_B	mut_sim_B
		#2	run_err_rate	mut_sim_B	timeout_rate	gold_sim_CB	mut_sim_B	run_err_rate	mut_sim_B	gold_sim_B
		#3	gold_sim_B	pass_rate	gold_sim_B	mut_sim_CB	mut_sim_CB	gold_sim_CB	gold_sim_CB	syn_err
	Comp	#1	gold_sim_CB	run_err_rate	black_count	gold_sim_CB	syn_err	run_err_rate	gold_sim_CB	mut_sim_B
		#2	timeout_rate	mut_sim_CB	timeout_rate	run_err_rate	timeout_rate	pass_rate	black_count	gold_sim_CB
		#3	mut_sim_CB	gold_sim_B	gold_sim_CB	gold_sim_CB	run_err_rate	syn_err	timeout_rate	black_count
Scenario	Clearer	#1	gold_sim_CB	gold_sim_CB	gold_sim_B	gold_sim_CB	timeout_rate	mut_sim_B	syn_err	gold_sim_B
		#2	gold_sim_B	mut_sim_CB	pass_rate	pass_rate	gold_sim_B	gold_sim_B	pass_rate	gold_sim_CB
		#3	syn_err	pass_rate	gold_sim_CB	mut_sim_B	syn_err	pass_rate	gold_sim_CB	pass_rate
	Improve	#1	timeout_rate	gold_sim_CB	mut_sim_B	mut_sim_B	gold_sim_B	syn_err	gold_sim_B	mut_sim_B
		#2	gold_sim_CB	black_count	pass_rate	gold_sim_B	run_err_rate	mut_sim_B	run_err_rate	gold_sim_B
		#3	black_count	syn_err	mut_sim_CB	syn_err	mut_sim_B	gold_sim_B	gold_sim_CB	gold_sim_CB
	Specify	#1	gold_sim_CB	mut_sim_CB	syn_err	pass_rate	pass_rate	gold_sim_B	mut_sim_B	mut_sim_CB
		#2	black_count	syn_err	black_count	syn_err	mut_sim_B	run_err_rate	mut_sim_CB	gold_sim_CB
		#3	syn_err	gold_sim_B	timeout_rate	mut_sim_B	syn_err	mut_sim_CB	pass_rate	run_err_rate

Fig. 7. Prompt effect analysis for the *instruction* group. For code metrics, Red indicates a negative effect received from the meta-prompt variable, while blue indicates a positive effect.

more thorny questions that involve third-party libraries and complex programming logic. This phenomenon also highlights the difference between datasets, suggesting that it is impossible to learn a universal causal graph and obtain general insights across all datasets.

7.2 RQ2: Analysis based on the Causal Graphs

In this section, we assess our analysis framework by employing it to learn the extent of the impact of meta-prompt variables on code metrics; this reflects how code metrics can be affected by prompts. We adopt Alg. 1 to achieve this objective, and present the results in Fig. 7. Each row in these figures represents a meta-prompt variable (see Fig. 4), with three sub-rows representing the top three code metrics that are mostly affected by the meta-prompt variable. Code metrics are highlighted in red and blue colors. The former implies that the meta-prompt variable has a negative effect on the code metric, whereas the latter indicates a positive effect. Note that we inspect the impact of meta-prompt variables separately to exhaustively comprehend how each variable influences the code metrics. However, we clarify that our analysis framework can be straightforwardly extended to analyze the combined effect of multiple meta-prompt variables.

The key observation from Fig. 7 is that each meta-prompt variable indeed influences distinct code metrics in different causal graphs. For example, in APPS, the mostly affected code metric for meta-prompt variable `Short` is `syn_err` in GPT-Neo, while it is `gold_sim_B` in GPT-3.5, and `pass_rate` in GPT-4. Nevertheless, on BigCodeBench, the mostly affected code metric changes to other metrics in most cases. We attribute this notable difference to the distinction between the learned graphs, as mentioned in Table 4's discussion. It is worth noting that even for the same model, the learned causal graph and the subsequent analysis can vary substantially across different datasets. On APPS, we can observe that the meta-prompt variables `Long` and `Formal` typically have a negative effect on the generated code, while `Short` and `Fluent` are beneficial. Nevertheless, this pattern is not as evident on BigCodeBench.

Although the causal graphs for different models and datasets can vary remarkably, there are some common patterns that can be observed. Compared to the *instruction* group, the effect of the *role* and *scenario* groups on the generated code tends to be more positive, particularly for more powerful models, i.e., GPT-3.5, GPT-4, and Qwen-2.5. In contrast, GPT-Neo fails to gain notable benefits from these two groups of rephrasing methods. We attribute this to the fact that the limited comprehension capability of GPT-Neo makes it difficult to understand the role and scenario, which are more abstract and complex compared to the straightforward rephrasing instructions. This result is consistent with previous research on prompt engineering [16, 79, 87], indicating that role-playing and scenario are efficient ways to enhance the performance of LLM-based code generation.

During the analysis, we also obtain several interesting findings from the perspective of the linguistic variables. For example, `root_det_var` occurs 23 times as the primarily responsible linguistic variables for the meta-prompt effect in GPT-Neo (see Alg. 1 for the detailed explanation of the responsible linguistic variables). This variable represents the value of the total number of unique determiners divided by the square root of the total number of determiners. In contrast, `simp_ttr` and features related to named entities occur most frequently for GPT series models. The former indicates the degree of lexical variation of the text, and the latter determines the complexity of the information contained in the text. We perceive that alterations in certain linguistic features may have a more profound effect on the generated code compared to others, and focusing on these features would be more effective in improving the performance of LLM-based code generation.

Findings: There are several interesting findings from the analysis that are worth mentioning.

(1) The learned causal graphs may vary substantially across different models and datasets, suggesting that it is essential to employ our analysis framework to conduct a customized analysis for each scenario, rather than simply generalizing the insights from one scenario to another.
(2) In general, using role-playing and scenario setting denotes a universal and effective way to improve the performance of LLM for models with sufficient capacity. However, for tiny models, using simple rephrasing instructions like "make it more concise" can often be more effective.

(3) Take advantage of the analysis pipeline illustrated in this work and watch out for the responsible linguistic variables. There may exist some linguistic variables that have a significant impact on the generated code. Focusing on these variables can be more effective in adjusting the prompt.

8 Downstream Application

In this section, we present a prospective downstream application on the basis of our approach in *improving the prompt to generate high-quality code*. Specifically, the procedure is based on the genetic algorithm which is widely used in the literature of search-based software engineering, such as test case generation [72]. The genetic algorithm is a metaheuristic search algorithm that mimics the process of natural selection. It operates on a population of candidate solutions and iteratively evolves the population to find the optimal solution. Below, we describe how these procedures are concretized in our context.

Fitness Function. This procedure aims to find the optimal rephrasing instructions that maximize the probability of generating high-quality code. First, as a straightforward solution, we may directly use the objective metric (e.g., BLEU score) in the fitness function and search for the optimal rephrasing instructions that maximize the objective metric. However, this solution is impractical in our context. The reason is that the objective metric requires instructing the LLM to generate code, which is costly and time-consuming. Furthermore, some metrics (e.g., `pass_rate` and `gold_sim_CB`) relies on the the ground truth or oracle, which is not always available, to evaluate the quality of generated code. In contrast, our causality analysis-based approach can estimate the expected value of the objective metric without involving the actual code generation and subsequent evaluation process. This way, the estimated value constitutes a lightweight surrogate for the objective metric, substantially reducing the cost. Specifically, based on the causal graph, we can estimate the expected value of the objective metric of any candidate solution by intervening on the rephrasing instructions (i.e., meta-prompt in Fig. 5). More importantly, the surrogate is also expected to be more stable than actual code generation, as it delineates the statistical expectation of the objective metric over all possible code generation outcomes, avoiding the infamous non-determinism issues in LLM [51].

Genetic Representation, Modification and Selection. We use a genetic algorithm to find the rephrasing instructions that can maximize a given objective metric. The instructions are represented as a binary string, with each bit indicating the selection status of the corresponding instruction. For example, "*make it short*" is represented as `100000`, and combining it with "*make it fluent*" is `110000`. The algorithm operates on these binary strings to find the best instructions. In each generation, each vector is evaluated using a predefined fitness function. Only the top-N candidates, based on fitness scores, move to the next generation (i.e., "survive"). The new generation is formed using these top-N vectors. Two standard operators in the genetic algorithm, Crossover and Mutation, are applied to create new offspring vectors from any two parent vectors.

Result. We apply the genetic algorithm to the APPS dataset and use the BLEU score as the objective metric. We evaluate our approach against the original prompt and the prompt rephrased by the best single instruction (among the "instruction" in Fig. 4, written by humans and optimized by GPT-4). Table 6 shows the results. We can see that our approach outperforms the best single instruction and the original prompt in most cases. Overall, we interpret the results as highly encouraging, suggesting that our algorithm has the high potential to improve prompts for generating high-quality code. We clarify, however, that the application is a proof-of-concept study in an idealized setting. The results may not generalize to real-world

Table 6. Evaluation of our rephrasing algorithm. Winner, whose BLEU score is the highest, is highlighted.

	gold_sim			mut_sim		
	Original	Single	Ours	**Original**	Single	Ours
GPT-Neo	0.7071	-19.33%	-13.65%	0.8030	+4.21%	+8.77%
GPT-3.5	0.5014	+4.77%	+10.59%	0.5072	+3.25%	+5.86%
GPT-4	0.4867	+9.33%	+15.47%	0.4971	+2.66%	+7.28%

scenarios considering the complexity and diversity of programming tasks. We believe presenting a full-fledged prompt optimization pipeline (on the basis of this paper), though promising, requires further study and validation, which we leave as future work.

9 Discussion

Generalizability and Extensibility. While the current technical pipeline is primarily applied over the task of generating Python3 code, it is extensible to other languages. Overall, our approach is generally language agnostic. Besides, our approach is also extensible to other rephrasing instructions. For example, more rephrasing instructions like "make it more creative" can be added. The only requirement is that the rephrasing instruction should be of reasonable difficulty for LLMs to understand and implement. To conclude, we believe that one of the primary advantages of our approach in comparison to existing methods is the availability of a single, off-the-shelf pipeline that can be seamlessly extended to user scenarios in order to obtain more specific and customized insights. Further, we believe that our approach can be extended to more sophisticated LLM-based coding scenarios, such as multi-agent collaborative coding [69], human-in-the-loop coding [68].

Threats to Validity. Our study is subject to the following threats to validity. First, our study primarily focuses limited LLMs (GPT-family and Qwen) and datasets (APPS, BigCodeBench). Second, merely a limited number of rephrasing instructions are investigated. Therefore, it is possible that our findings may not generalize to other settings. However, given the popularity of GPT-family models and the Python language, our findings could be representative for a wide range of scenarios. Given the extensibility discussed above, we believe that our methodology could be smoothly applied to other LLMs, datasets, and rephrasing instructions.

10 Related Work

We have discussed related work on LLM-based code generation in Sec. 2. In this section, we discuss related work on causality analysis and its application in software engineering.

Theoretical Causality Analysis. Causality analysis has been studied in statistics and machine learning for decades [54, 55]. Algorithms from various categories typically employ different strategies for learning the causal graphs [15, 61, 66, 70]. Recently, gradient-based algorithms [85, 88] have been proposed, seeking to recover the data generation process while adhering to acyclicity constraints. These advancements in the theoretical aspect of causality analysis are orthogonal to our work. In this paper, we focus on the application of it on LLM-based code generation.

Causality Analysis in Software Engineering. It is shown that causality analysis is applicable to various software engineering tasks [63], including debugging [17, 19], root cause analysis [30, 35], data race detection [26], and also deep neural network testing and repairing [32, 67, 86]. Besides, causality analysis has been used for understanding empirical software engineering data and uncover the underlying causal relations, such as [21, 71]. In this paper, we advocate the application of causality analysis for understanding the role of distinct prompts in LLM-based code generation.

11 Conclusion

In line with the prosperous adoption of LLMs in code generation, this paper proposes a novel causality analysis-based approach to systematically analyze the relations between the LLM input prompts and the generated code. Our solution facilitates the evaluation and explanation of LLM-based code generation, and provides actionable insights to improve the quality of the LLM-generated code by properly calibrating the prompt. Our work can serve as a roadmap for users to comprehend and improve the quality of LLM-generated code.

References

[1] [n. d.]. Black - The uncompromising code formatter. https://black.readthedocs.io/en/stable/

[2] [n. d.]. Semgrep — Find bugs and enforce code standards. https://semgrep.dev/

[3] [n. d.]. Tree-sitter. https://tree-sitter.github.io/tree-sitter/

[4] 2022. EconML: A Python Package for ML-Based Heterogeneous Treatment Effects Estimation. https://github.com/microsoft/EconML.

[5] 2023. ChatGPT. https://chat.openai.com/chat.

[6] 2023. Rephrasing Tool - QuillBot AI. https://quillbot.com/.

[7] 2023. Research Artifact. https://anonymous.4open.science/r/CALL-E7F2/.

[8] Arshiya Aggarwal, Jiao Sun, and Nanyun Peng. 2022. Towards Robust NLG Bias Evaluation with Syntactically-diverse Prompts. In *Findings of the Association for Computational Linguistics: EMNLP 2022*. 6022–6032.

[9] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).

[10] Teodora Baluta, Shiqi Shen, S Hitarth, Shruti Tople, and Prateek Saxena. 2022. Membership inference attacks and generalization: A causal perspective. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 249–262.

[11] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in NeurIPS*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc.

[12] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[13] Victor Chernozhukov, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James Robins. 2016. Double/debiased machine learning for treatment and causal parameters. *arXiv preprint arXiv:1608.00060* (2016).

[14] Cheng-Han Chiang and Hung-yi Lee. 2023. Can large language models be an alternative to human evaluations? *arXiv preprint arXiv:2305.01937* (2023).

[15] James Cussens, Matti Järvisalo, Janne H Korhonen, and Mark Bartlett. 2017. Bayesian network structure learning with integer programming: Polytopes, facets and complexity. *JAIR* 58 (2017), 185–229.

[16] Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. 2023. Improving Factuality and Reasoning in Language Models through Multiagent Debate. *arXiv preprint arXiv:2305.14325* (2023).

[17] Clemens Dubslaff, Kallistos Weis, Christel Baier, and Sven Apel. 2022. Causality in configurable software systems. In *Proceedings of the 44th International Conference on Software Engineering*. 325–337.

[18] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated repair of programs from large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1469–1481.

[19] Anna Fariha, Suman Nath, and Alexandra Meliou. 2020. Causality-guided adaptive interventional debugging. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 431–446.

[20] Joseph L Fleiss. 1971. Measuring nominal scale agreement among many raters. *Psychological bulletin* 76, 5 (1971), 378.

[21] Carlo A Furia, Richard Torkar, and Robert Feldt. 2023. Towards Causal Analysis of Empirical Software Engineering Data: The Impact of Programming Languages on Coding Competitions. *arXiv preprint arXiv:2301.07524* (2023).

[22] Luca Giamattei, Antonio Guerriero, Roberto Pietrantuono, and Stefano Russo. 2024. Causal reasoning in Software Quality Assurance: A systematic review. *Information and Software Technology* (2024), 107599.

[23] Cordell Green. 1981. Application of theorem proving to problem solving. In *Readings in Artificial Intelligence*. Elsevier, 202–222.

[24] Sumit Gulwani, Oleksandr Polozov, Rishabh Singh, et al. 2017. Program synthesis. *Foundations and Trends® in Programming Languages* 4, 1-2 (2017), 1–119.

[25] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring Coding Challenge Competence With APPS. *NeurIPS* (2021).

[26] Chun-Hung Hsiao, Satish Narayanasamy, Essam Muhammad Idris Khan, Cristiano L Pereira, and Gilles A Pokam. 2017. Asyncclock: Scalable inference of asynchronous event causality. *ACM SIGPLAN Notices* 52, 4 (2017), 193–205.

[27] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. 2024. Qwen2. 5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).

[28] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436* (2019).

[29] Yoichi Ishibashi, Danushka Bollegala, Katsuhito Sudoh, and Satoshi Nakamura. 2023. Evaluating the Robustness of Discrete Prompts. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. 2365–2376.

[30] Zhenlan Ji, Pingchuan Ma, and Shuai Wang. 2023. Perfce: Performance debugging on databases with chaos engineering-enhanced causality analysis. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1454–1466.

[31] Zhenlan Ji, Pingchuan Ma, Shuai Wang, and Yanhui Li. 2023. Causality-aided trade-off analysis for machine learning fairness. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 371–383.

[32] Zhenlan Ji, Pingchuan Ma, Yuanyuan Yuan, and Shuai Wang. 2023. CC: Causality-Aware Coverage Criterion for Deep Neural Networks. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1788–1800.

[33] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515* (2024).

[34] Wenxiang Jiao, Wenxuan Wang, Jen-tse Huang, Xing Wang, and Zhaopeng Tu. 2023. Is ChatGPT a good translator? A preliminary study. *arXiv preprint arXiv:2301.08745* (2023).

[35] Brittany Johnson, Yuriy Brun, and Alexandra Meliou. 2020. Causal testing: understanding defects' root causes. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 87–99.

[36] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. 2022. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems* 35 (2022), 21314–21328.

[37] Bruce W Lee, Yoo Sung Jang, and Jason Lee. 2021. Pushing on Text Readability Assessment: A Transformer Meets Handcrafted Linguistic Features. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 10669–10686.

[38] Bruce W. Lee and Jason Lee. 2023. LFTK: Handcrafted Features in Computational Linguistics. In *Proceedings of the 18th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2023)*. 1–19.

[39] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-Level Code Generation with AlphaCode. *arXiv preprint arXiv:2203.07814* (2022).

[40] Zongjie Li, Chaozheng Wang, Zhibo Liu, Haoxuan Wang, Dong Chen, Shuai Wang, and Cuiyun Gao. 2023. Cctest: Testing and repairing code completion systems. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1238–1250.

[41] Zongjie Li, Chaozheng Wang, Zhibo Liu, Haoxuan Wang, Dong Chen, Shuai Wang, and Cuiyun Gao. 2023. CCTEST: Testing and Repairing Code Completion Systems. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 1238–1250.

[42] Zongjie Li, Chaozheng Wang, Pingchuan Ma, Chaowei Liu, Shuai Wang, Daoyuan Wu, Cuiyun Gao, and Yang Liu. 2024. On Extracting Specialized Code Abilities from Large Language Models: A Feasibility Study. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 74, 13 pages.

[43] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *ACM Comput. Surv.* (2023).

[44] Yue Liu, Chakkrit Tantithamthavorn, Yonghui Liu, and Li Li. 2024. On the reliability and explainability of language models for program generation. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), 1–26.

[45] Lars Lorch, Jonas Rothfuss, Bernhard Schölkopf, and Andreas Krause. 2021. Dibs: Differentiable bayesian structure learning. *Advances in Neural Information Processing Systems* 34 (2021), 24111–24123.

[46] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664* (2021).

[47] Xiaofei Lu. 2010. Automatic analysis of syntactic complexity in second language writing. *International journal of corpus linguistics* 15, 4 (2010), 474–496.

[48] Zohar Manna and Richard J Waldinger. 1971. Toward automatic program synthesis. *Commun. ACM* 14, 3 (1971), 151–165.

[49] Brady Neal. 2015. Introduction to Causal Inference. (2015).

[50] R OpenAI. 2023. GPT-4 technical report. *arXiv* (2023), 2303–08774.

[51] Shuyin Ouyang, Jie M Zhang, Mark Harman, and Meng Wang. 2023. LLM is Like a Box of Chocolates: the Non-determinism of ChatGPT in Code Generation. *arXiv preprint arXiv:2308.02828* (2023).

[52] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 311–318.

[53] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the keyboard? assessing the security of github copilot's code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 754–768.

[54] Judea Pearl. 2009. *Causality*. Cambridge university press.

[55] Jonas Peters, Dominik Janzing, and Bernhard Schölkopf. 2017. *Elements of causal inference: foundations and learning algorithms*. The MIT Press.

[56] KC Sreedharan Pillai. 1955. Some new test criteria in multivariate analysis. *The Annals of Mathematical Statistics* (1955), 117–121.

[57] Matt Post. 2018. A Call for Clarity in Reporting BLEU Scores. In *Proceedings of the Third Conference on Machine Translation: Research Papers*. Association for Computational Linguistics, Belgium, Brussels, 186–191. https://www.aclweb.org/anthology/W18-6319

[58] Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with" gradient descent" and beam search. *arXiv preprint arXiv:2305.03495* (2023).

[59] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* (2020).

[60] Victor Sanh, Albert Webson, Colin Raffel, Stephen H. Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Arun Raja, Manan Dey, M Saiful Bari, Canwen Xu, Urmish Thakker, Shanya Sharma Sharma, Eliza Szczechla, Taewoon Kim, Gunjan Chhablani, Nihal V. Nayak, Debajyoti Datta, Jonathan Chang, Mike Tian-Jian Jiang, Han Wang, Matteo Manica, Sheng Shen, Zheng Xin Yong, Harshit Pandey, Rachel Bawden, Thomas Wang, Trishala Neeraj, Jos Rozen, Abheesht Sharma, Andrea Santilli, Thibault Févry, Jason Alan Fries, Ryan Teehan, Teven Le Scao, Stella Biderman, Leo Gao, Thomas Wolf, and Alexander M. Rush. 2022. Multitask Prompted Training Enables Zero-Shot Task Generalization. In *ICLR 2022, Virtual Event, April 25-29, 2022*.

[61] Mauro Scanagatta, Cassio P de Campos, Giorgio Corani, and Marco Zaffalon. 2015. Learning Bayesian Networks with Thousands of Variables.. In *NIPS*. 1864–1872.

[62] Taylor Shin, Yasaman Razeghi, Robert L Logan IV, Eric Wallace, and Sameer Singh. 2020. AutoPrompt: Eliciting Knowledge from Language Models with Automatically Generated Prompts. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 4222–4235.

[63] Julien Siebert. 2023. Applications of statistical causal inference in software engineering. *Information and Software Technology* (2023), 107198.

[64] Karan Singhal, Shekoofeh Azizi, Tao Tu, S Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, et al. 2023. Large language models encode clinical knowledge. *Nature* (2023), 1–9.

[65] Armando Solar-Lezama. 2008. *Program synthesis by sketching*. University of California, Berkeley.

[66] Peter Spirtes, Clark N Glymour, Richard Scheines, and David Heckerman. 2000. *Causation, prediction, and search*.

[67] Bing Sun, Jun Sun, Long H Pham, and Jie Shi. 2022. Causality-based neural network repair. In *Proceedings of the 44th International Conference on Software Engineering*. 338–349.

[68] Wannita Takerngsaksiri, Jirat Pasuksmit, Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Ruixiong Zhang, Fan Jiang, Jing Li, Evan Cook, Kun Chen, and Ming Wu. 2024. Human-In-the-Loop Software Development Agents. *arXiv preprint arXiv:2411.12924* (2024).

[69] Yashar Talebirad and Amirhossein Nadiri. 2023. Multi-agent collaboration: Harnessing the power of intelligent llm agents. *arXiv preprint arXiv:2306.03314* (2023).

[70] Ioannis Tsamardinos, Laura E Brown, and Constantin F Aliferis. 2006. The max-min hill-climbing Bayesian network structure learning algorithm. *Machine learning* 65, 1 (2006), 31–78.

[71] Masateru Tsunoda and Sousuke Amasaki. 2017. On software productivity analysis with propensity score matching. In *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 436–441.

[72] Sapna Varshney and Monica Mehrotra. 2013. Search based software test data generation for structural testing: a perspective. *ACM SIGSOFT Software Engineering Notes* 38, 4 (2013), 1–6.

[73] Chaozheng Wang, Yuanhang Yang, Cuiyun Gao, Yun Peng, Hongyu Zhang, and Michael R Lyu. 2022. No more fine-tuning? an experimental evaluation of prompt tuning in code intelligence. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 382–394.

[74] Ruoke Wang, Zongjie Li, Chaozheng Wang, Yang Xiao, and Cuiyun Gao. 2024. NAVRepair: Node-type Aware C/C++ Code Vulnerability Repair. *arXiv preprint arXiv:2405.04994* (2024).

[75] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, and Denny Zhou. 2022. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *CoRR* abs/2203.11171 (2022).
[76] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859* (2021).
[77] Bolin Wei, Ge Li, Xin Xia, Zhiyi Fu, and Zhi Jin. 2019. Code generation as a dual task of code summarization. *Advances in neural information processing systems* 32 (2019).
[78] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. 2022. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682* (2022).
[79] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903* (2022).
[80] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C Schmidt. 2023. A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382* (2023).
[81] Wai Kin Wong, Huaijin Wang, Zongjie Li, Zhibo Liu, Shuai Wang, Qiyi Tang, Sen Nie, and Shi Wu. 2023. Refining decompiled c code with large language models. *arXiv preprint arXiv:2310.06530* (2023).
[82] Yuxin Xiao, Paul Pu Liang, Umang Bhatt, Willie Neiswanger, Ruslan Salakhutdinov, and Louis-Philippe Morency. 2022. Uncertainty quantification with pre-trained language models: A large-scale empirical analysis. *arXiv preprint arXiv:2210.04714* (2022).
[83] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).
[84] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. 2023. Large language models as optimizers. *arXiv preprint arXiv:2309.03409* (2023).
[85] Yue Yu, Jie Chen, Tian Gao, and Mo Yu. 2019. DAG-GNN: DAG structure learning with graph neural networks. In *International Conference on Machine Learning*. PMLR, 7154–7163.
[86] Mengdi Zhang and Jun Sun. 2022. Adaptive fairness improvement based on causality analysis. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 6–17.
[87] Yifan Zhang, Jingqin Yang, Yang Yuan, and Andrew Chi-Chih Yao. 2023. Cumulative Reasoning With Large Language Models. *arXiv preprint arXiv:2308.04371* (2023).
[88] Xun Zheng, Bryon Aragam, Pradeep K Ravikumar, and Eric P Xing. 2018. Dags with no tears: Continuous optimization for structure learning. *Advances in Neural Information Processing Systems* 31 (2018).
[89] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2022. Large language models are human-level prompt engineers. *arXiv preprint arXiv:2211.01910* (2022).
[90] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. 2024. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. *arXiv preprint arXiv:2406.15877* (2024).
[91] Daniel M Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. 2019. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593* (2019).

Received 2024-10-31; accepted 2025-03-31